

JMPM

Relatório de Status dos Componentes

DATA

04/10/2026

ARQUIVO

COMPONENT_STATUS.md

gRPC embarcado, Smart Link real e auditoria de 27 docs TDN (atualizado 2026-08-23)

Status Atual: Funcional e testado ponta a ponta (cliente e servidor reais)

Auditoria de conformidade contra 27 documentos TDN de referência (diretivas de preprocessador `#command` / `#xcommand` / `#translate` / `#xtranslate`, criptografia, classes "Não Visual"/"TLPP - Classes úteis", família RPO), seguida da implementação de todos os gaps reais encontrados e de duas rodadas de acompanhamento pedidas depois: "implemente o framework gRPC real (<https://grpc.io/>)" no lugar de um stub, e — ao descobrir via código-fonte real do Protheus 12.1.2510 (TechFin `backoffice.techfin.util.smartlink.tlpp`, RH Insights `rh.sigagpe.insights.sendendcalc.tlpp`) que apps reais usam `FwTotvsLinkClient` (HTTP+fila) em vez de `tGrpc` diretamente — implementar essa classe também.

O Que Funciona:

- ✓ `Argon2id` (função) e `tpbkdf2` (classe OOP completa, SHA1-SHA3-512): via `golang.org/x/crypto/argon2` / `pbkdf2`
- ✓ `tHashMap` classe OOP (`New/Set/Get/Del/List/Clean/nStatus`): interoperava com a API funcional histórica `HMNew` / `HMSet` / ...
- ✓ `tJsonParser` (`New/Json_Hash/Json_Parser/json_ok`): parsing JSON real, popula um `tHashMap` real quando pré-criado pelo chamador
- ✓ `tUnicode` (`Normalize` NFC/NFD/NFKC/NFKD, `ConvertEncoding`): via `golang.org/x/text/unicode/norm`
- ✓ `TFtpClient`: cliente FTP real (RFC 959, PASV) — testado upload/download com round-trip de conteúdo verificado contra um servidor FTP real
- ✓ `Resource2File` / `GetPatchFile` / `SRCheckSourceSignature`: completam a família "Manipulação de RPO"
- ✓ **2 bugs reais corrigidos no motor** `#command`: wild match marker `<*x*>`, extended-expression match marker `<(x)>` (nome do marcador nunca era casado), dumb stringify `#<x>` e smart stringify `<(x)>` (nunca reconhecidos) — a suíte de 500 testes que levou o motor a "100%" não exercitava essas 4 formas
- ✓ `tGrpc` (cliente gRPC real): `google.golang.org/grpc` de verdade — conexão HTTP/2 real, descoberta de serviço/método via **gRPC Server Reflection** em runtime (sem `.proto` local nenhum) e invocação dinâmica via `dynamicpb`
- ✓ `GRPCServer` (servidor gRPC real, simétrico ao `WSRestServer`): `AddMethod` / `Serve` expõem `User Function` como RPCs unárias com Server Reflection habilitada — testado com um cliente Go genuíno que descobre e invoca via reflection, zero stub gerado em tempo de compilação. Achado real durante o teste: o handler inicialmente usava `jsonMapToAdvplObject` (maiusculiza chaves, correto para `MCPServer`) em vez de `jsonToAdvplValue` (preserva case, correto para JSON externo genérico com acesso por colchete) — corrigido
- ✓ `FwTotvsLinkClient` (cliente Smart Link real, `net/http`): OAuth2 `client_credentials` real; endpoint/credenciais/tenant via `ADVPP_SMARTLINK_BASEURL` / `TOKENURL` / `CLIENTID` / `CLIENTSECRET` / `TENANTID` — sem configurar, falha honesta de rede, nunca sucesso simulado

Notas de Implementação:

- `pkg/vm/argon2_native.go`, `pkg/vm/pbkdf2_native.go`, `pkg/vm/tunicode_native.go`, `pkg/vm/ftpclient_native.go`, `pkg/vm/tgrpc_native.go`, `pkg/vm/grpcserver_native.go`, `pkg/vm/totvslinkclient_native.go`, `pkg/vm/jsonparser_native.go` (novos); `pkg/vm/matrizhashmap_native.go`, `pkg/vm/rpo_native.go`, `pkg/preprocessor/commands.go` (estendidos)
- Novas dependências: `google.golang.org/grpc` v1.71.0, `google.golang.org/protobuf` v1.36.4 (compatíveis com go1.24 — versões mais novas exigem go1.25); `golang.org/x/crypto` e `golang.org/x/text` já eram dependências existentes
- `go build / go vet` cross-compile Linux/Windows/macOS limpos; `go test ./...` 20/20 pacotes verdes, sem regressão
- Detalhe técnico completo (assinaturas, limitações reais que permanecem, notas de confiança ⓘ INFERIDO onde a TDN não publica subpágina de detalhe) em `docs/tdn-known-limitations.md`

Primitivas de TUI de baixo nível (atualizado 2026-08-02)

Status Atual: Funcional e coberto por teste de integração

Além do `TerminalUIProvider` (diálogos prontos: `MSDIALOG` / `FWGetText` / `FWMenuSelect`), o AdvPP agora expõe primitivas visuais de baixo nível para um programa AdvPL compor a própria TUI — estilo opencode/Claude Code (caixas com borda, markdown renderizado no terminal, tela alternativa, streaming de LLM token a token). Ver `pkg/vm/ui_render.go` e seção 4.4.1 do `GUIA_DO_DESENVOLVEDOR_PARA_ADVPP.md`.

O Que Funciona:

- ✓ `UiBox` / `UiStreamBox` / `UiStreamReset` (lipgloss): caixa com borda arredondada e título; a variante `Stream` apaga e redesenha por cima a cada chamada (altura rastreada em `VM.lastBoxLines`), dando o efeito de "cartão crescendo ao vivo" sem raw-mode de teclado
- ✓ `UiMarkdown` (glamour): negrito/itálico/listas/código/títulos para ANSI, estilo "dark" fixo — mesma cautela contra auto-deteção de tema via OSC 11 (pode travar em terminal/multiplexer que não responde) já aplicada ao lipgloss em `pkg/ui/terminal.go`
- ✓ `UiAltScreenEnter` / `UiAltScreenExit`: tela alternativa (buffer do vim/less/htop) com handler de Ctrl+C que restaura antes de encerrar — sem isso um Ctrl+C durante o app travaria o terminal do usuário na tela alternativa
- ✓ `UiTermWidth`: largura real do terminal, cai num default configurável quando stdout não é TTY
- ✓ `ConOutRaw`: escreve sem newline — texto em streaming (deltas de LLM) na mesma linha
- ✓ `ConIn` **distingue EOF real de linha vazia**: agora devolve Nil (não `" "`) quando stdin fecha de verdade (Ctrl+D, pipe esgotado) — um REPL consegue sair do loop em vez de reimprimir o prompt pra sempre
- ✓ `ProcRun`: executa um processo filho (sem shell) com callback síncrono por linha de stdout/stderr — consumo de NDJSON em streaming (ex.: CLI de um LLM) direto de dentro do AdvPL
- ✓ `JsonObject.FromJson` **é um parser real** (era stub que sempre devolvia Nil sem parsear nada): objetos aninhados viram `JsonObject`, arrays viram `Array` 1-based, `null` → Nil; `.F.` (não erro) em JSON inválido
- ✓ **Paridade Windows**: toda saída que pode conter ANSI passa por

`stdoutW` (`go-colorable` no Windows, `os.Stdout` puro no Linux/macOS) — sem isso `cmd.exe` /PowerShell sem `ENABLE_VIRTUAL_TERMINAL_PROCESSING` imprimiriam os escapes literalmente

Notas de Implementação:

- `pkg/vm/ui_render.go` : as 7 natives de TUI (`registerUiRenderNatives`)
- `pkg/vm/natives.go` : `stdoutW` , `CONOUTRAW` , `PROCRUN` , `CONIN` (EOF → Nil)
- `pkg/vm/vm.go` : `VM.lastBoxLines` , `jsonToAdvplValue` , `JsonObject:FromJson` real
- Nova dependência: `github.com/charmbracelet/glamour` (+ `go-colorable`)
- Teste: `cmd/advplc/tui_natives_test.go` + fixture `tests/tui_natives_test.prw` — exercita as 7 natives de TUI, `ConOutRaw` , `ProcRun` (contra o próprio binário buildado, portátil entre SOs), `FromJson` (válido/inválido/aninhado) e o EOF de `ConIn`

Executável Standalone — Console/TUI (atualizado 2026-07-29)

Status Atual: Funcional e coberto por teste de integração

Até esta data, `advplc build` compilava e o binário resultante "rodava" sem erro mesmo quando a interação real (login, menus, `FWMBrowse`) nunca acontecia de fato — nenhum teste automatizado exercitava esse caminho, só o smoke test 100%-console (`TestBuildStandaloneSmoke` , zero prompts). Os dois problemas abaixo existiam desde a introdução do `build` e só foram descobertos ao tentar usar um app de verdade (e-Gov) interativamente:

O Que Funciona:

- ✓ **Console interativo real** (`pkg/ui/terminal.go` , `TerminalUIProvider`): `FWGetText` / `FWMenuSelect` / `Msg*` leem/escrevem no terminal de verdade quando stdin é um TTY — antes, sem nenhum `UIProvider` anexado no modo headless, esses natives nunca tocavam stdin e devolviam o default silenciosamente (login "passava direto" sem pedir nada)
- ✓ **TUI real via `huh` + `lipgloss`** : formulários com borda, seleção por seta, tema escuro fixo (sem depender de query OSC 11 ao terminal, que pode travar em ambientes que não respondem)
- ✓ **`FWMBrowse` funciona no console**, não só em `advplc serve` — listagem em tabela (largura adaptada ao terminal, com aviso de campos ocultos), Novo/Editar/Excluir com formulário multi-campo
- ✓ **Deteção console-vs-GUI corrigida**: a heurística antiga checava `bc.Classes` , que só registra classes *declaradas* (`class X from Y`) — nunca detectava `FWMBrowse():New()` (toda instanciação real de classe builtin). Trocada por varredura real do bytecode (`OP_CALL_NATIVE` para `FWGetText/FWMenuSelect`, `OP_NEW_INSTANCE` para `FWMBrowse/MSDIALOG/etc`)
- ✓ **`ADVPP_FORCE_GUI=1`** : força a janela Fyne mesmo rodando de um terminal — para um app cujo autor prefere GUI como padrão (ex.: e-Gov, GesCon), sem mudar o comportamento padrão (console-se-tem-TTY) de outros programas AdvPP
- ✓ Correção de bug correlato em `FWMBrowse` : o fallback de colunas sem entrada no SX3 expunha `R_E_C_N_O_` / `R_E_C_D_E_L_` (chave interna) como campo editável — um Novo/Editar sobrescrevia o próprio rowid e o INSERT/UPDATE seguinte falhava com "datatype mismatch"

Notas de Implementação:

- `pkg/ui/terminal.go` : `TerminalUIProvider` (implementa `vm.UIProvider` e `vm.BrowseUI`)

- `pkg/compiler/stub_template.go`: decisão console-vs-GUI, `ADVPP_FORCE_GUI`
- `pkg/vm/browse.go`: exclusão de `R_E_C_N_O_` / `R_E_C_D_E_L_` do fallback sem SX3
- Teste de integração real (builda o binário, roda sob um pseudo-terminal de verdade, digita/navega/confirma exatamente como um humano):
`cmd/advplc/build_standalone_interactive_test.go`
 (`TestBuildStandaloneInteractive`), fixture `tests/standalone_interactive_test.prw`
 — roda nos 3 SOs do CI (Linux/macOS via `creack/pty`, Windows via `github.com/UserExistsError/conpty`, ConPty real em Go puro sobre `golang.org/x/sys/windows`, sem cgo), abstraído em `ptysession_posix_test.go` / `ptysession_windows_test.go` — mesmo teste, mesmas asserções, cobertura real nas três plataformas que o release publica

Adendo (2026-08-01): `advplc build --gui` fixa a decisão console-vs-GUI em tempo de compilação (a heurística acima descrita continua sendo o comportamento padrão sem a flag). Diferença prática do `ADVPP_FORCE_GUI` já documentado: `--gui` também linka o binário Windows no subsistema GUI (`-H=windowsgui`, `pkg/compiler/standalone.go:goBuildArgs`), então nenhuma janela de console aparece nem por um instante — `ADVPP_FORCE_GUI` sozinho não muda o subsistema. Obrigatório para programas com `MSDIALOG`.

Componentes UI

Status Atual: Renderização Visual Implementada

O compilador AdvPP agora inclui renderização completa de widgets Fyne para todos os componentes UI.

O Que Funciona:

- ✓ Estruturas de dados de componentes (TButton, TGet, TComboBox, TCheckBox, etc.)
- ✓ Estruturas de diálogo (Dialog, MenuBar, ToolBar, StatusBar)
- ✓ Propriedades de componentes (X, Y, Width, Height, Label, Value, etc.)
- ✓ Framework de tratamento de eventos (onChange, onClick, onGotFocus, onLostFocus)
- ✓ **Renderização de widgets Fyne para todos os componentes**
- ✓ **Renderização de TButton**
- ✓ **Renderização de TGet (entrada de texto)**
- ✓ **Renderização de TComboBox**
- ✓ **Renderização de TCheckBox**
- ✓ **Renderização de TLabel**
- ✓ **Renderização de MenuBar**
- ✓ **Renderização de ToolBar**
- ✓ **Renderização de StatusBar**
- ✓ **Renderização de view de formulário com conteúdo scrollable**
- ✓ Diálogos Fyne básicos (MsgInfo, MsgStop, MsgAlert, MsgYesNo)
- ✓ Componentes UI da IDE (CodeEditor, OutputConsole, FileTree)

O Que NÃO Funciona:

- ✗ Execução de eventos de componentes (manipuladores definidos mas não conectados)
- ✗ Atualizações dinâmicas de componentes (sem two-way binding)

Notas de Implementação:

- Componentes são definidos como structs Go em `pkg/mvc/view.go`
- Renderização Fyne implementada em `pkg/ui/renderer.go`
- Executável de teste visual: `./ui-test` (em `cmd/ui-test/`)
- Renderização completa de componentes agora funcional

Recursos REST 2.0

Status Atual: Funcional (estilo anotações) / Apenas Parsing (DSL clássico WSRESTFUL)

O compilador AdvPP sobe um servidor HTTP REST **real** (`pkg/rest` + classe nativa `WSRestServer` , mesmo padrão arquitetural do `MCPServer`) para o estilo moderno TLPP de anotações (`@Get` / `@Post` / `@Put` / `@Patch` / `@Delete` sobre `User Function`). O DSL clássico `WSRESTFUL ... WSMETHOD ... ENDWSRESTFUL` continua **apenas parseado** — ver limitação abaixo.

O Que Funciona:

- ✓ **Servidor HTTP real** (`net/http` puro, sem CGO/dependências): classe `WSRestServer` — `New()` , `AddRoute(verbo, path, funcao)` , `Serve(porta)`
- ✓ **Auto-discovery de rotas via anotação**: toda `User Function` anotada com `@Get("/path")` / `@Post(...)` / `@Put(...)` / `@Patch(...)` / `@Delete(...)` vira rota automaticamente ao criar o `WSRestServer`
- ✓ **Path params** (`/clientes/{id}`) via roteador nativo do Go 1.22+ (`http.ServeMux`), populados no objeto de argumento da função AdvPL (`http.ServeMux`), populados no objeto de argumento da função AdvPL
- ✓ **Query string e corpo JSON** decodificados e mesclados no objeto de argumento (`oParam:CAMPO`) — corpo tem precedência sobre path params, que tem precedência sobre query string
- ✓ **Dispatch real para a função AdvPL** via VM isolada por requisição (mesmo mecanismo do `MCPServer` / `StartJob` : `v.RunFunction`), banco e bytecode compartilhados
- ✓ **Resposta JSON automática** do retorno da função (200), erro vira 500, path inexistente vira 404, verbo não registrado num path existente vira 405
- ✓ Registro manual de rota via `AddRoute` (cobrir casos onde a anotação não é usada)
- ✓ Sintaxe JSON inline, métodos `JsonObject` (`toJson`, `hasProperty`, `getJsonText`), serialização/deserialização JSON

O Que NÃO Funciona (limitações conhecidas):

- ✗ **DSL clássico** `WSRESTFUL <nome> ... WSMETHOD <verbo> ... PATH "..." ... ENDWSRESTFUL` : o parser (`parseWSClient`) em `pkg/parser/parser.go` reconhece a sintaxe e monta um `ast.ClassDecl` , mas descarta o verbo HTTP e a cláusula `PATH` ao fazer isso — só o nome do `WSMETHOD` sobrevive como protótipo. Além disso, a implementação real do método (`WSMETHOD ... WSSERVICE <classe>`) é um método de instância, e `v.RunFunction` (usado pelo dispatch HTTP) só chama funções top-level, não métodos de classe — precisaria criar a instância do serviço a cada requisição. Nenhuma das duas partes foi implementada: exigiria cirurgia de parser (capturar verbo+path como antes) e um caminho de dispatch novo (instanciar + chamar método) só para esse estilo, sem um caso de uso real nos corpora validados para justificar o esforço agora. Workaround: reescrever o serviço no estilo anotações (`@Get` / `@Post` sobre `User Function`), que já é 100% funcional, ou registrar a rota manualmente via `AddRoute` .
- ✗ **Geração de WSDL / cliente REST** (`WSCIENT` consumindo serviço externo) — fora de escopo, é consumo e não exposição de API
- ✗ **Controle fino de código HTTP de resposta pela função AdvPL** (sempre 200 em sucesso / 500 em erro — sem equivalente a `SetLegacySuccess` / `GetHTTPCode` do lado servidor)

Notas de Implementação:

- Servidor: `pkg/rest/rest.go` (stdlib `net/http`, roteador nativo do Go 1.22+, sem dependências externas)
- Ponte VM: `pkg/vm/rest_native.go` (classe nativa `WSRestServer`, conversão JSON ↔ `advplrt.Value`, auto-discovery via `FunctionInfo.Annotations` do bytecode)
- Fixture de teste: `tests/rest_server_test.prw`; teste de integração Go (builda o binário, sobe o servidor, faz requisições HTTP reais): `cmd/advplc/rest_integration_test.go` (`TestRestServerFixture`)

Construção de Serviços

Status Atual: Parcial

O Que Funciona:

- ✓ Parsing de sintaxe WSCLIENT/WSSTRUCT/WSRESTFUL
- ✓ Definições de protótipo WSMETHOD
- ✓ Definições de campos WSDATA
- ✓ Metadados de serviço (DESCRIPTION, NAMESPACE)
- ✓ Criação e manipulação de objetos JSON

O Que NÃO Funciona:

- ✗ Geração de código WSDL
- ✗ Geração de código de cliente REST
- ✗ Invocação de serviço
- ✓ **Integração de cliente HTTP nativa:** `FWHttpGet` / `FWHttpPost` / `FWHttpPut` / `FWHttpPatch` / `FWHttpDelete` + `FWHttpRequestBody` / `FWHttpRequestStatus` / `FWHttpRequestError` com suporte a certificado PKCS#12 (.pfx/.p12), timeout 30s, TLS com verificação. Testes de integração com servidor HTTP local. Adicionado v2.0.1.

Banco de Dados e Multi-thread (atualizado 2026-07-08)

Status Atual: Funcional

- ✓ **Banco SQLite compartilhado:** todas as ferramentas (advplc, adveditor, advpp-ide) resolvem o mesmo banco via `shared.ResolveDatabasePath` (flag → `ADVPP_DB` → config real em disco → `./advpp.db` local do diretório de trabalho, criado automaticamente)
- ✓ **Driver 100% Go** (modernc.org/sqlite, sem CGO) com WAL + busy_timeout
- ✓ **Natives de banco conectados ao VM:** DBSelectArea, DBSeek, DBSkip, RecCount, FieldGet/FieldPut etc. operam sobre o banco real
- ✓ **StartJob(cFunc, cEnv, IWait, params...):** execução em VM isolado (goroutine), síncrona ou assíncrona, com conexão própria ao banco
- ✓ **FWGridProcess:** pool de threads com backpressure (SetThreadGrid, CallExecute, StopExecute, IsFinished, meters, SaveLog)
- ✓ **advplc check paralelo:** N arquivos com 1 worker por CPU
- ✓ **Renderer web (advplc serve):** PO-UI embutido no binário; console, diálogos, FWMBrowse → po-table com dicionário SX3, po-dynamic-form, MSDIALOG legado por heurística de grade e hot reload (--watch)
- ✓ **Motor de inferência LLM (classe `LLM`):** modelos GGUF I2_S (BitNet/Falcon3-1.58bit) via `pkg/llm`, 100% Go sem CGO, com kernel SIMD AVX2 em amd64 (fallback escalar em qualquer outra arquitetura), validado token a token contra o `llama.cpp` de referência. Desde v4.0.0, também decodifica pesos Q4_K/Q6_K e reconhece `general.architecture = "minicpm"` — validado executando o modelo real MiniCPM5-2B-Q4_K_M via `advplc run`, com saída batendo com o

`llama.cpp` de referência

- ✓ **Conectividade real multi-provider (classe `DbConnection` , desde **v4.0.0**):** `PostgreSQL` / `Oracle` / `SQL Server` via drivers 100% Go sem CGO (`pgx` , `go-ora` , `go-mssqldb`). `DBSetDriver("TOPCONN")` roteia `DBUseArea` /RDD pra conexão remota ativa (`RemoteSQLEngine`); SQL direto (`TCGenQry` / `TCSqlToArr`) já funciona assim que `DbConnection:Connect()` tem sucesso. `TCLINK` /SQLite local inalterados. Validado com round-trip real de CRUD contra um PostgreSQL real
- ✓ **Servidor MCP nativo (classe `MCPServer`):** JSON-RPC 2.0 real sobre stdio via `pkg/mcp` (initialize/tools.list/tools.call), expõe funções AdvPL como tools — execução real; validado com o SDK oficial em Python do MCP
- ✓ **Servidor REST nativo (classe `WSRestServer`):** HTTP real via `pkg/rest` (`net/http` puro), auto-discovery de rotas por anotação (`@Get` / `@Post` / `@Put` / `@Patch` / `@Delete`) ou registro manual via `AddRoute` , path params, dispatch para a função AdvPL via `v.RunFunction` — mesmo padrão do `MCPServer` ; o DSL clássico `WSRESTFUL` / `WSMETHOD` continua só parseado (ver "Recursos REST 2.0")
- ⚠ Locks de registro (RecLock/MsUnlock) são no-ops — sem controle de concorrência em escrita entre processos

Resumo

Recurso	Status	Notas
Componentes UI	✓ Completo	Renderização Fyne completa implementada
Diálogos UI	✓ Completo	MsgInfo, MsgStop, MsgAlert, MsgYesNo funcionam
Parsing REST	✓ Completo	Sintaxe totalmente parseada
Execução REST (anotações <code>@Get/@Post</code>)	✓ Funcional	Servidor HTTP real (<code>WSRestServer</code>), dispatch para a função AdvPL
Execução REST (DSL <code>WSRESTFUL/WSMETHOD</code>)	✗ Nenhum	Apenas parsing — ver "Recursos REST 2.0"
Anotações <code>@Get/@Post/@Put/@Patch/@Delete</code>	✓ Executadas	Viram rota HTTP automaticamente
Suporte JSON	✓ Completo	Sintaxe inline e JsonObject funcionam
Construção de Serviços	⚠ Parcial	Parseado, não gerado

Compatibilidade com IDE

Status Atual: 100% Compatível

O compilador AdvPP e todos os componentes UI são totalmente compatíveis com a IDE AdvPP.

Resultados de Testes

- ✓ Todos os arquivos de teste em `tests/*.prw` passam (`make test`)
- ✓ Componentes MVC funcionam no contexto da IDE
- ✓ Integração de provider UI funciona
- ✓ Saída do compilador com componentes UI funciona
- ✓ Execução da VM com renderização UI funciona
- ✓ Funções de diálogo (MsgInfo, MsgStop, MsgAlert, MsgYesNo) funcionam

- ✓ Suporte JSON funciona
- ✓ Funções nativas funcionam
- ✓ Estruturas de controle funcionam
- ✓ Arrays funcionam
- ✓ Funções de string funcionam

Teste de Integração IDE

```
./advplc run tests/ide_integration_test.prw
```

Todos os testes passaram - 100% de compatibilidade com IDE verificada.

Teste de Renderização UI

Para testar a renderização UI:

```
go build -o ui-test ./cmd/ui-test
./ui-test
```

Isso exibirá uma janela com:

- TLabel (título)
- TGet (entrada de texto)
- TComboBox (dropdown)
- TCheckBox (checkbox)
- TButton (botões)
- ToolBar (topo)
- StatusBar (fundo)

Recomendações

1. **Para Componentes UI:** Implementar sistema de renderização de widgets Fyne ou documentar como apenas dados
2. **Para REST:** DSL clássico `WSRESTFUL` / `WSMETHOD` ainda precisa de captura de verbo+path no parser e dispatch via instância de classe (hoje `v.RunFunction` só chama funções top-level) — sem caso de uso real nos corpora para priorizar agora
3. **Para Serviços:** Adicionar geração de código ou integração de cliente HTTP
4. **Documentação:** Atualizar README para separar claramente recursos "parseados" de "executados"